

EC 551 – Advanced Digital Design with Verilog and FPGAs

Lecture 5 Simulation and Test

Prof. Douglas Densmore

Announcements

- Post your final groups to Piazza in the discussion forum (1 post per group). Follow the template post.

Analog Simulation



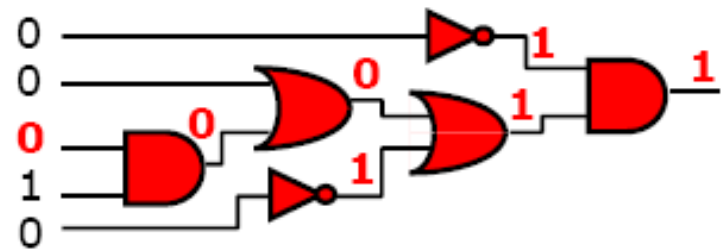
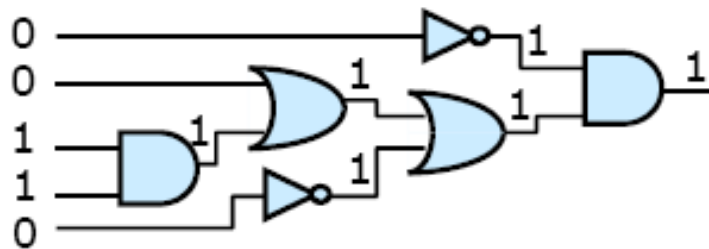
- ❑ Divide “time” into slices
- ❑ Update information in whole circuit at each slice
- ❑ Used by SPICE

- ❑ Allows detailed modeling of current and voltage
- ❑ Computationally intensive and slow

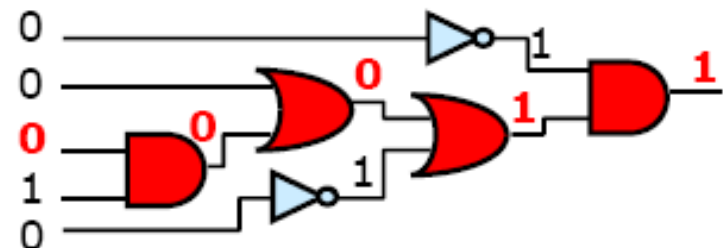
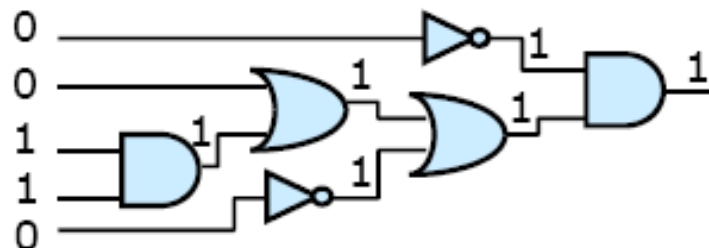
- ❑ Don't need this level of detail for most digital logic simulation

Digital Simulation

- Could update every signal on input change



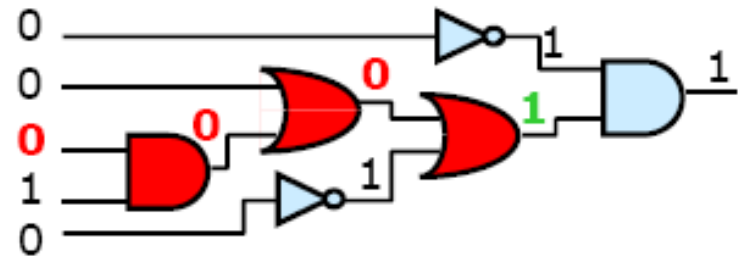
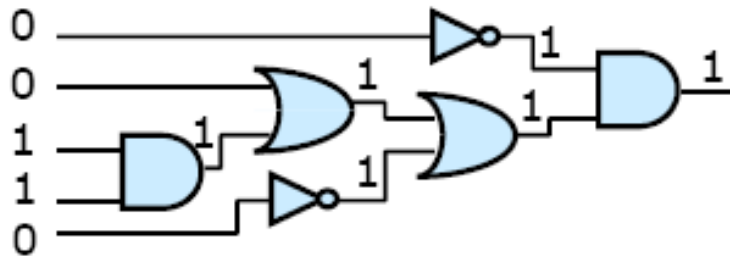
- Could update just the full path on input change



- Don't even need to do that much work!

Simulation

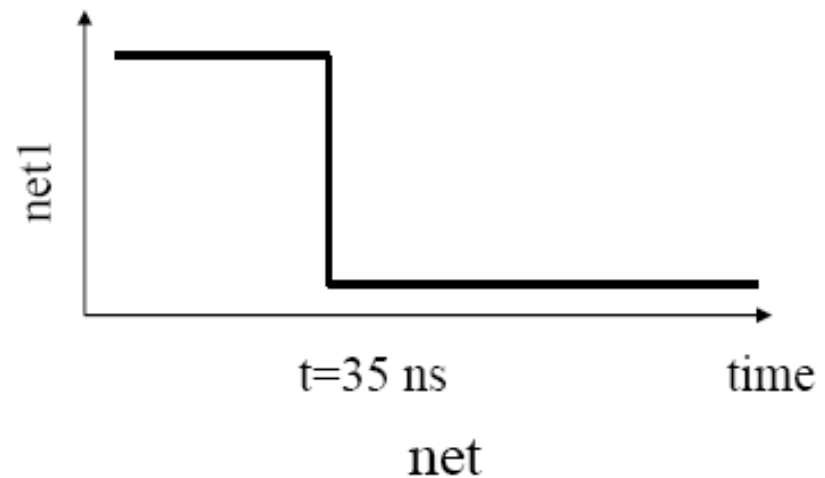
- Update only if changed



- Some circuits are very large
 - Updating every signal => very slow simulation
 - Event-driven simulation is much faster!

Event-driven simulation

- An **event** is a change in a net's value.
- An event has two components:
 - **value;**
 - **time.**

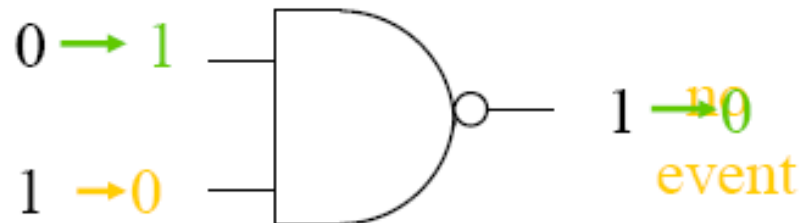


net1=0 @ 35 ns

event

Events on a gate

- ❑ Propagate events only when nets change value.
- ❑ If an input change doesn't cause an output change, no event is propagated.

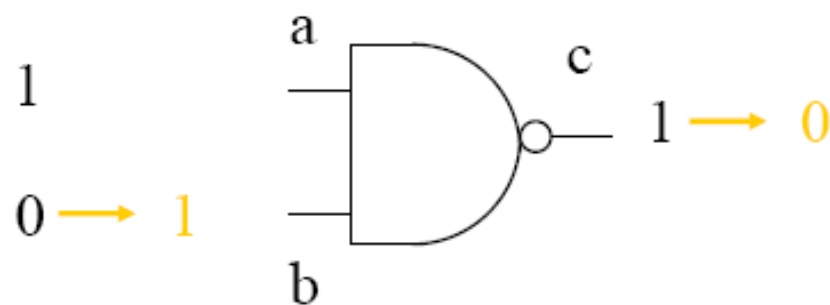


Timewheel

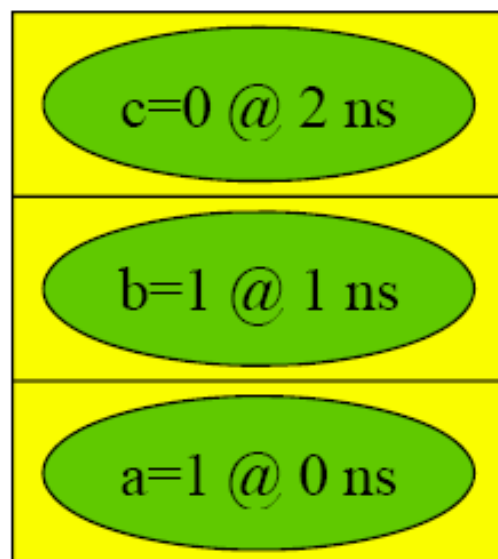


- ❑ The timewheel is a **data structure** in the simulator that efficiently determines the order of events processed.
- ❑ Events are placed on the timewheel in **time order**.
- ❑ Events are taken **out of the head of the timewheel** to process them in order.

Timewheel operation



netlist



time ↑

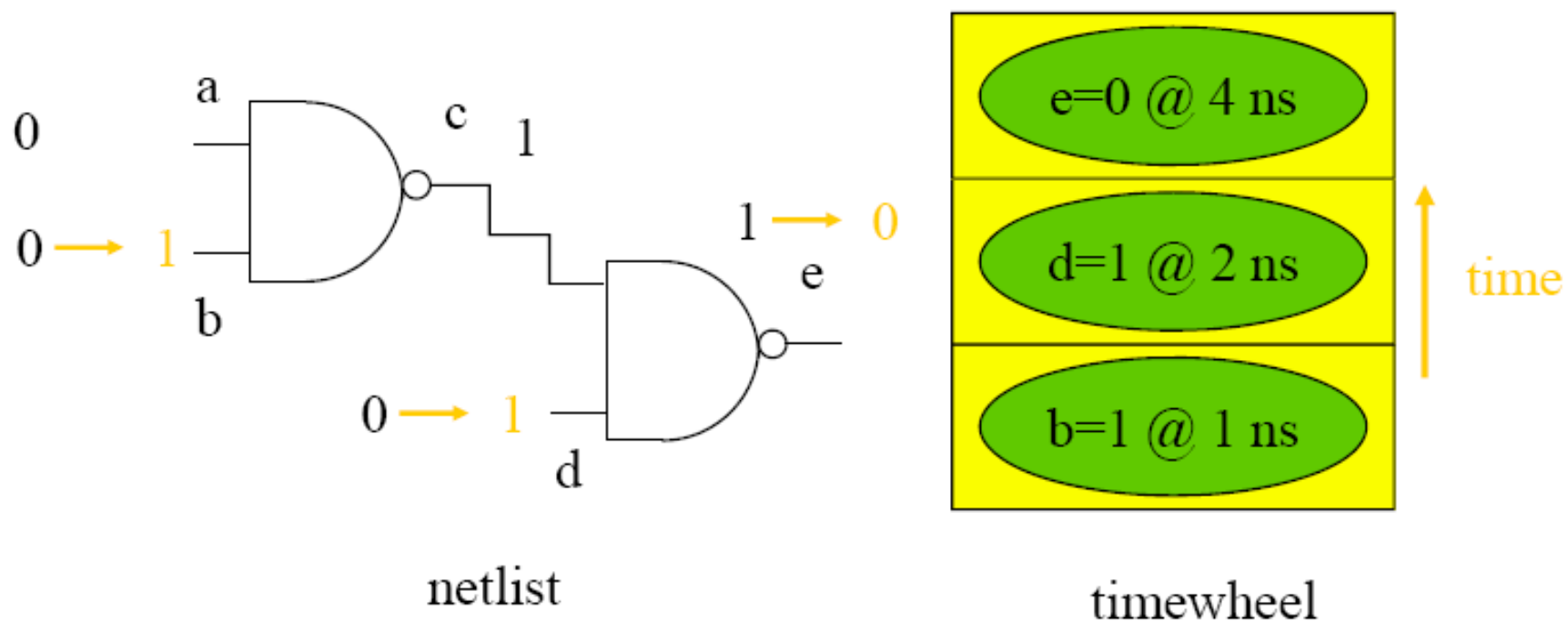
timewheel

Order of evaluation



- ❑ Order of evaluation is important.
 - **Causality** must be obeyed.
- ❑ Evaluating events in the wrong order can cause inaccurate results.

Order of evaluation example



Event-Driven Simulation

- When an input to the simulating circuit changes, put it on a “changed” list
- Loop while the “changed” list isn’t empty:
 - Remove a signal from the “changed” list
 - For each sink of the signal
 - Recompute its new output(s)
 - For any output(s) that have changed value, add that signal to the “changed” list
- When the “changed” list is empty, keep simulation results until next input change

Simulation vs. programming



- ❑ Simulation tags computations with times.
 - Must know **when signals change** to properly simulate hardware.
- ❑ **Simulation is parallel.**
 - Many statements can execute at the same (simulation) time.
 - **Just like hardware.**

HDLs language constructs

- ❑ Must be able to define component types.
 - A model may be behavioral or structural.
- ❑ May be able to define abstract data types.
 - A wire may carry an enumerated value.
 - Multi-valued simulation may be defined using abstract data types.
- ❑ May be able to define modules to control the scope of names.

Four-Value Logic

- A single bit can have one of FOUR values
 - 0 Numeric 0, logical FALSE
 - 1 Numeric 1, logical TRUE
 - x Unknown or ambiguous value
 - z No value (high impedance)

- Why **x**?
 - Could be a conflict, could be lack of initialization

- Why **z**?
 - Nothing driving the signal
 - Tri-states

The x and z values

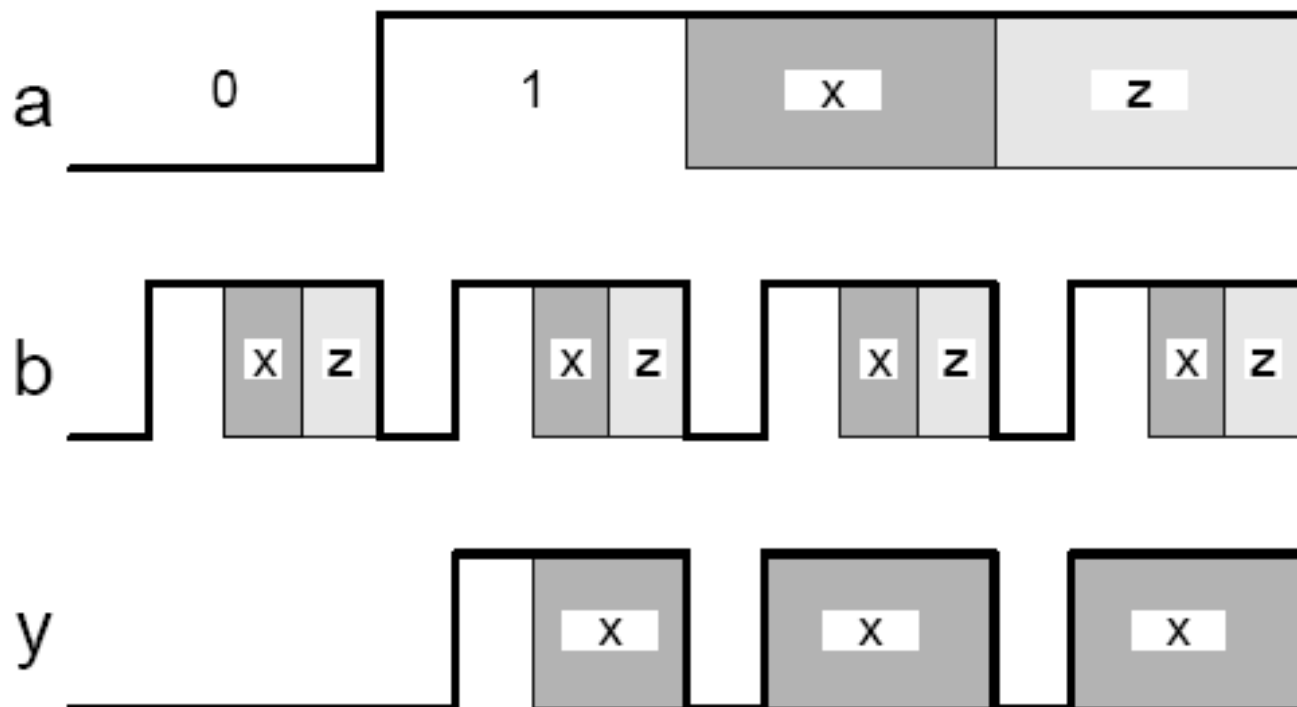
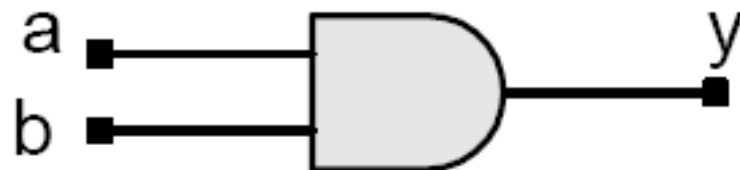
□ IN SIMULATION

- Can detect x or z using special comparison operators
- **x is useful to see:**
 - Uninitialized signals
 - Conflicting drivers to a wire
 - Undefined behavior

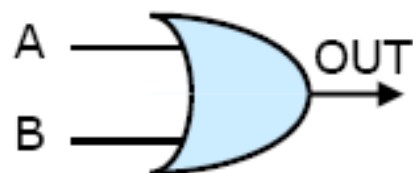
□ IN REALITY

- Cannot detect x or z
- No actual 'x' – electrically just isn't 0, 1, or z
- Except for some uninitialized signals, x is bad!
 - Multiple strong conflicting drivers => short circuit
 - Weak signals => circuit can't operate, unexpected results
- z means nothing driving signal (tri-state)

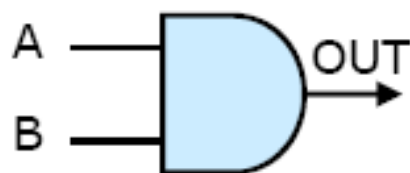
Example: 4-Valued Logic



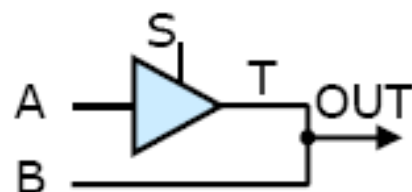
Resolving 4-Value Logic



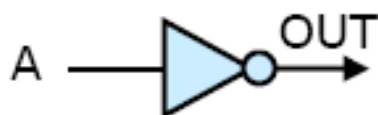
A	B	OUT
0	0	0
0	1	1
1	1	1
0	x	x
0	z	x
1	x	1
1	z	1



A	B	OUT
0	0	0
0	1	0
1	1	1
0	x	0
0	z	0
1	x	x
1	z	x



S	A	T	B	OUT
0	0	z	z	z
0	1	z	x	x
0	x	z	1	1
0	z	z	0	0
1	0	0	1	x
1	0	0	z	0
1	1	1	z	1
1	x	x	z	x
1	z	x	0	x



A	0	1	x	z
OUT	1	0	x	x

Wired Logic



- ❑ The family of nets includes the types **wand** and **wor**
 - A **wand** net type resolves multiple driver as wired-and logic
 - A **wor** net type resolves multiple drivers as wor logic

- ❑ The family of nets includes **supply0** and **supply1**
 - **supply0** has a fixed logic value of 0 to model a ground connection
 - **supply1** has a fixed logic value of 1 to model a power connection

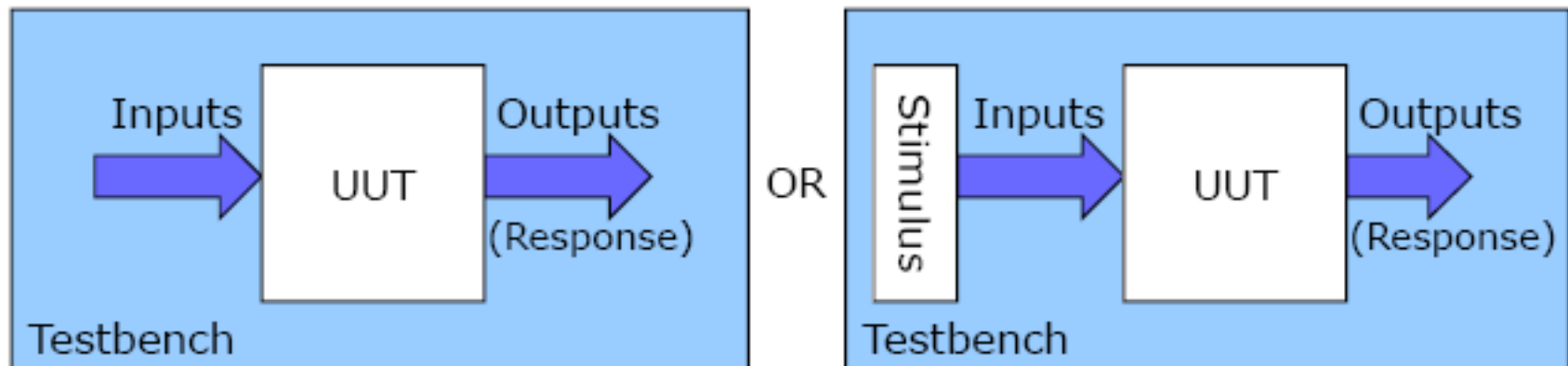
Example: simulation



- ❑ Apply stimulus/test vectors.
- ❑ Look at response/output vectors.
- ❑ Can't exhaustively simulate but we can exercise the module.
- ❑ Simulation before synthesis is faster and easier than simulating the mapped design.
 - Sometimes want to simulate the mapped design.

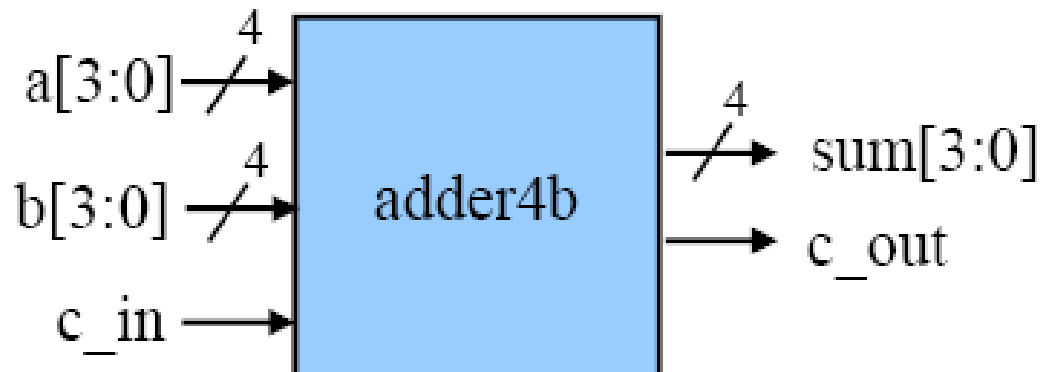
Simulation of Verilog

- ❑ Need to verify your design
 - "Unit Under Test" (UUT)
- ❑ Use a "testbench"!
 - **Special Verilog module with no ports**
 - Generates or routes inputs to the UUT
 - Outputs information about the results

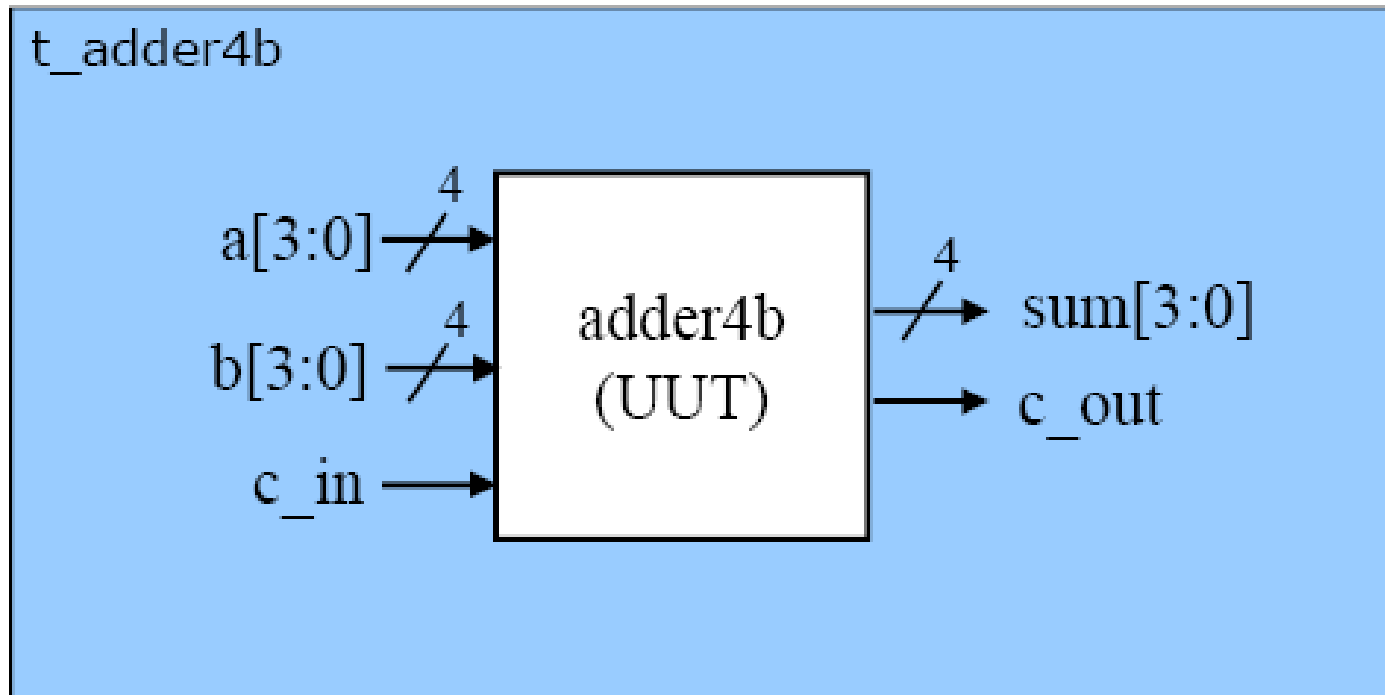


Simulation Example

```
module adder4b (sum, c_out, a, b, c_in);  
  input      [3:0]    a, b;  
  input      c_in;  
  output     [3:0]    sum;  
  output     c_out;  
  assign {c_out, sum} = a + b + c_in;  
endmodule
```



Simulation Example



- Testbenches frequently named `t_<UUT name>`

Signal Generators

- ❑ Statements in a behavior may be grouped in **begin** ... **end** blocks
- ❑ Execution begins at $t_{sim} = 0$
- ❑ # delay control operator temporarily suspends execution of a behavior
- ❑ The operator = denotes procedural assignment (also called **blocking assignment**)

MODELING TIP

Use procedural assignments to describe stimulus patterns in a testbench.

Testbench template

Consider the following template as a [guide for simple testbenches](#):

```
module t_DUTB_name ();           // substitute the name of the UUT
  reg ...;           // Declaration of register variables for primary inputs of the UUT
  wire ...;          // Declaration of primary outputs of the UUT
  parameter         time_out = // Provide a value

  UUT_name M1_instance_name ( UUT ports go here);
  initial $monitor ( ); // Specification of signals to be monitored and displayed as text
  initial #time_out $stop; // (Also $finish) Stopwatch to assure termination of simulation
  initial           // Develop one or more behaviors for pattern generation and/or
                    // error detection

  begin
    // Behavioral statements generating waveforms
    // to the input ports, and comments documenting
    // the test. Use the full repertoire of behavioral
    // constructs for loops and conditionals.

  end
endmodule
```

Types Of Delays



❑ Inertial Delay (Gates)

- Suppresses pulses shorter than delay amount
- In reality, gates need to have inputs held a certain time before output is accurate
- This models that behavior

❑ Transport Delay (Nets)

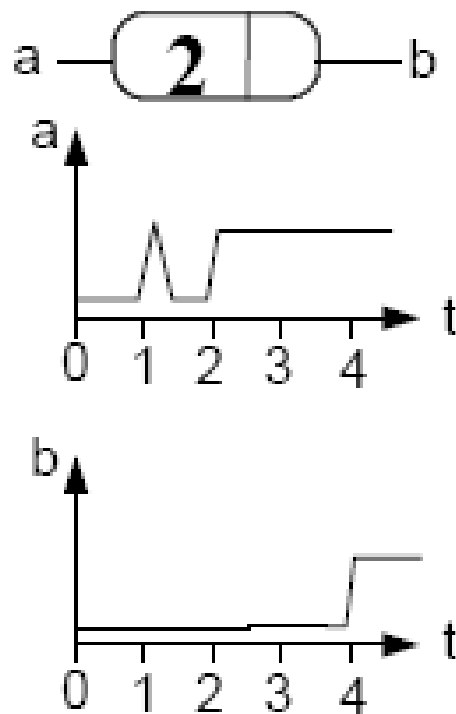
- “Time of flight” from source to sink
- Short pulses transmitted

❑ Not critical for most of class

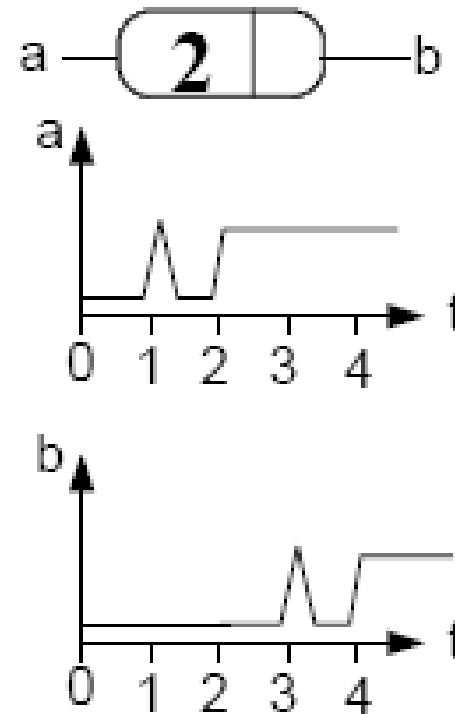
- May need to know when debugging
- Good to know for building very accurate simulation

Delay models

□ Inertial delay suppresses short pulses (width less than the propdelay value)

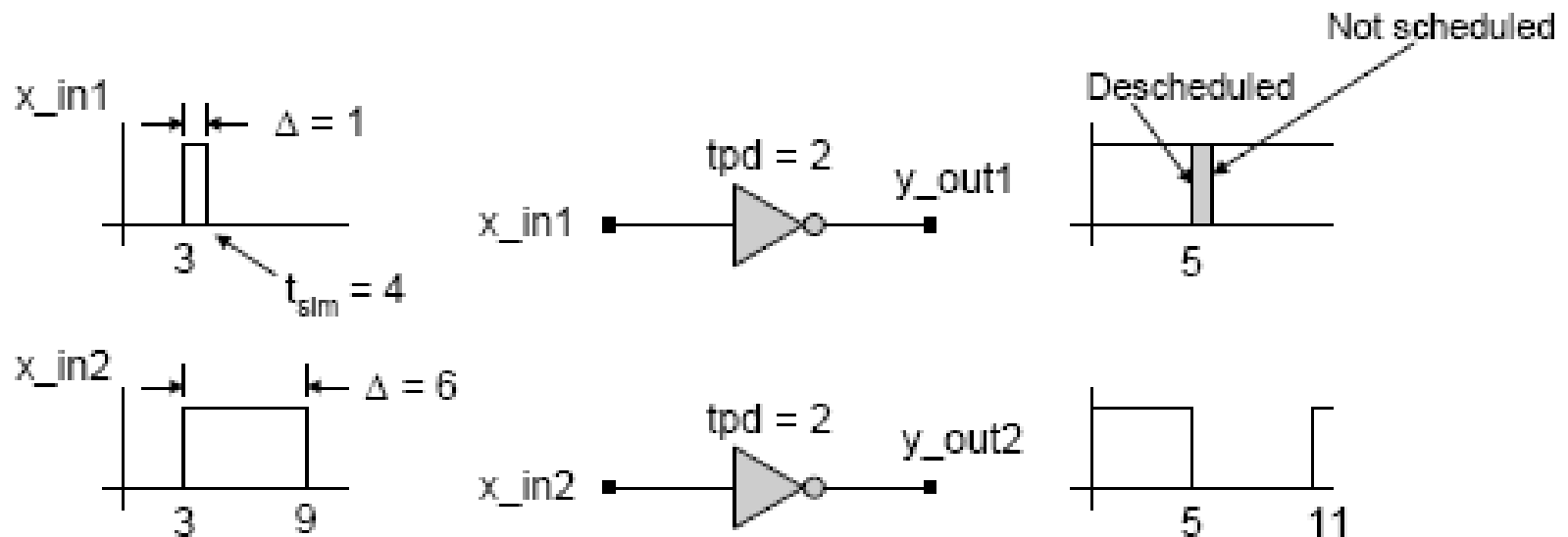


Inertial delay



Pure delay (e.g. transport delay)

Event de-scheduling caused by an inertial delay



Note: The falling edge of x_{in1} occurs before the response to the rising edge occurs.

Delay Examples

- `wire #5 net_1;` `// 5 unit transport delay`

- `and #4 (z_out, x_in, y_in);` `// 4 unit inertial delay`
- `assign #3 z_out = a & b;` `// 3 unit inertial delay`

- `wire #2 z_out;` `// 2 unit transport delay`
- `and #3 (z_out, x_in, y_in);` `// 3 for gate, 2 for wire`

- `wire #3 c;` `// 3 unit transport delay`
- `assign #5 c = a & b;` `// 5 for assign, 3 for wire`

Propagation Delay

- ❑ Gate propagation delay specifies the time between an input change and the resulting output change
- ❑ Transport delay describes the time-of-flight of a signal transition
- ❑ Verilog uses an inertial delay model for gates and transport delay for nets

MODELING TIP

All primitives and nets have a default propagation delay of 0.

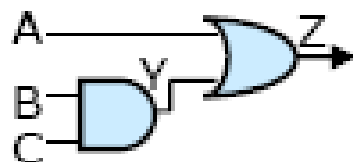
Zero Delay vs. Unit Delay

- ❑ When no timing controls specified: zero delay
 - Unrealistic – even electrons take time to move
 - OUT is updated same time A and/or B change:
and(OUT, A, B)
- ❑ Unit delay often used
 - Not accurate either, but closer...
 - “Depth” of circuit does affect speed!
 - Easier to see how changes propagate through circuit
 - OUT is updated 1 “unit” after A and/or B change:
and #1 A0(OUT, A, B);

Timing Controls For Simulation

- ❑ Can put “delays” in a Verilog design
 - Gates, wires, even behavioral statements!
- ❑ SIMULATION
 - Used to approximate “real” operation while simulating
 - Used to control testbench
- ❑ SYNTHESIS
 - Synthesis tool **IGNORES these timing controls**
 - Cannot tell a gate to wait 1.5 nanoseconds!
 - **Delay is a result of physical properties!**
 - Only timing (easily) controlled is on clock-cycle basis
 - Can tell synthesizer to **attempt** to meet cycle-time restriction

Zero/Unit Delay Example



Zero Delay:
Y and Z change
at same "time"
as A, B, and C!

Unit Delay:
Y changes 1 unit
after B, C

Unit Delay:
Z changes 1 unit
after A, Y

Zero Delay					
T	A	B	C	Y	Z
0	0	0	0	0	0
1	0	0	1	0	0
2	0	1	0	0	0
3	0	1	1	1	1
4	1	0	0	0	1
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1
8	0	0	0	0	0
9	0	0	1	0	0
10	0	1	0	0	0
11	0	1	1	1	1
12	1	0	0	0	1
13	1	0	1	0	1
14	1	1	0	0	1
15	1	1	1	0	1

Unit Delay					
T	A	B	C	Y	Z
0	0	1	0	x	x
1	0	1	0	0	x
2	0	1	0	0	0
3	0	1	1	0	0
4	0	1	1	1	0
5	0	1	1	1	1
6	1	0	0	1	1
7	1	0	0	0	1
8	1	1	1	0	1
9	1	1	1	1	1
10	1	0	0	1	1
11	1	0	0	0	1
12	0	1	0	0	1
13	0	1	0	0	0
14	0	1	1	0	0
15	0	1	1	1	0
16	0	1	1	1	1

Delays In Testbenches



- ❑ Single testbench tests many possibilities
 - Need to examine each case separately
 - Spread them out over "time"

- ❑ Use to generate a clock signal
 - Example later

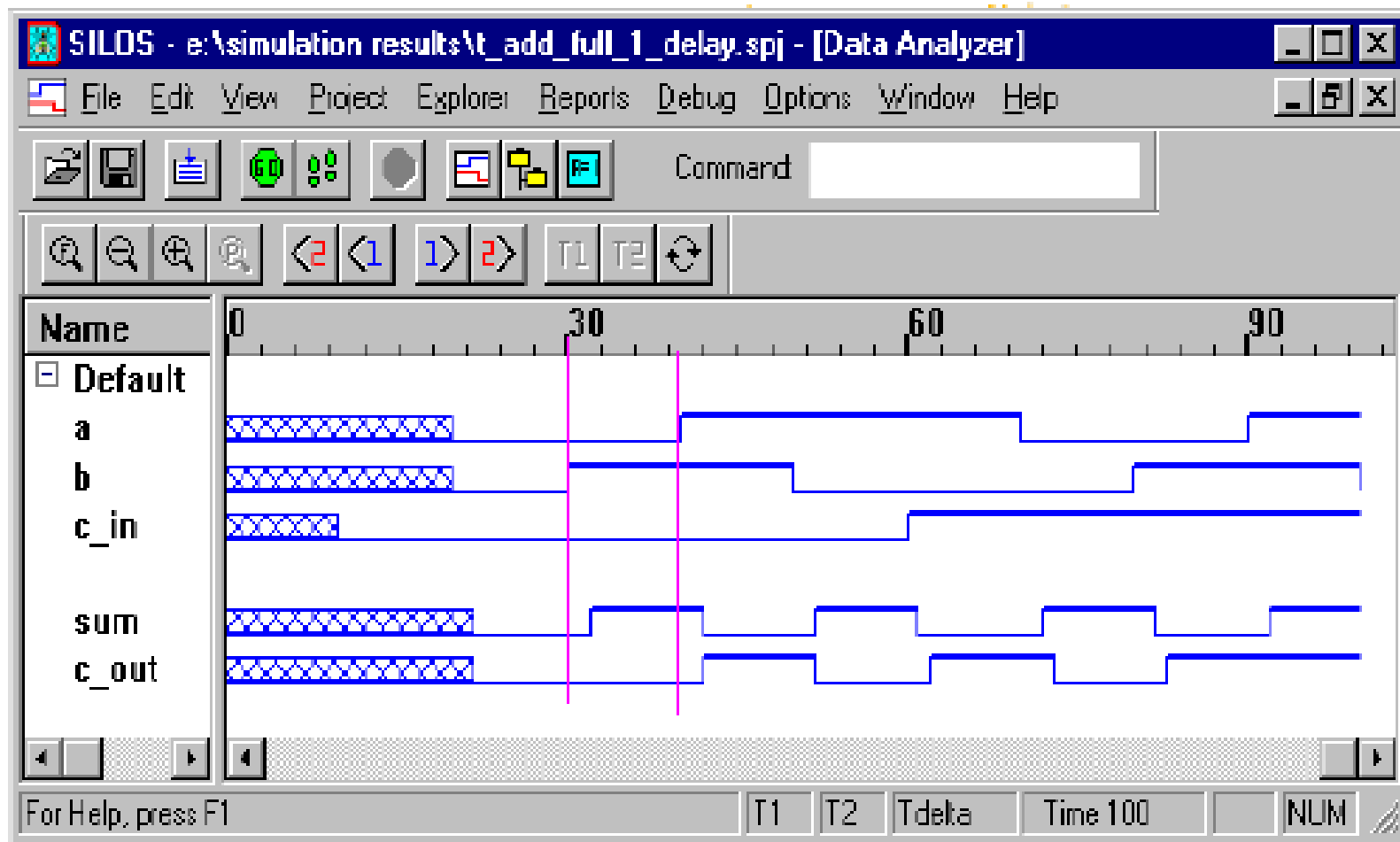
Example: Propagation Delay

Unit-delay simulation reveals the chain of events

```
module Add_full (sum, c_out, a, b, c_in);  
  output    sum, c_out;  
  input     a, b, c_in;  
  wire      w1, w2, w3;  
  Add_half  M1 (w1, w2, a, b);  
  Add_half  M2 (sum, w3, w1, c_in);  
  or        #1 M3 (c_out, w2, w3);  
endmodule
```

```
module Add_half (sum, c_out, a, b);  
  output    sum, c_out;  
  input     a, b;  
  xor       #1 M1 (sum, a, b);    // single delay value format  
  and       #1 M2 (c_out, a, b);  // others are possible  
endmodule
```

Results of unit-delay simulation



Simulation Example

```
`timescale 1ns /1ns                // time_unit/time_precision
module t_adder4b;
    reg[8:0] stim;                    // inputs to UUT are regs
    wire[3:0] S;                      // outputs of UUT are wires
    wire C4;

    // instantiate UUT
    adder4b(S, C4, stim[8:5], stim[4:1], stim[0]);

    // stimulus generation
    initial begin
        stim = 9'b0000000000;        // at 0 ns
        #10 stim = 9'b1111000001;    // at 10 ns
        #10 stim = 9'b0000011111;    // at 20 ns
        #10 stim = 9'b1111000010;    // at 30 ns
        #10 stim = 9'b0000111110;    // at 40 ns
        #10 $stop;                   // at 50 ns – stops simulation
    end
endmodule
```

Behaviors for Abstract Models



- ❑ Verilog has three types of behaviors for composing abstract models of functionality
 - Continuous assignment (Keyword: **assign**) - later
 - Single pass behavior (Keyword: **initial**) – Note: only use in testbenches
 - Cyclic behavior (Keyword: **always**) - later
- ❑ Single pass and cyclic behaviors execute procedural statements like a programming language
- ❑ The procedural statements execute sequentially
- ❑ A single pass behavior expires after the last statement executes
- ❑ A cyclic behavior begins executing again after the last statement executes

Testbench Requirements



- ❑ Instantiate the unit being tested (UUT)
- ❑ Provide input to that unit
 - Usually a number of different input combinations!
- ❑ Watch the “results” (outputs of UUT)
 - Can watch ModelSim Wave window...
 - Can print out information to the screen or to a file

Representation of Numbers

- Sized numbers specify the number of bits that are to be stored for a value
- Base specifiers:
 - b or B binary
 - d or D decimal (default)
 - o or O octal
 - h or H hexadecimal
- Example (in-class exercise):
- *8'ha* –

- Note Unsized numbers are stored as integers (at least 32 bits)

Writing a Test Bench (1)

```
initial
begin
    A=0; B=0;
    #10    A=1;
    #20    A=0;    B=1;
end
```

- The block is enclosed between **begin** and **end**. At time=0, A and B are set to 0. 10 time units later, A is changed to 1. 20 time units later (at t=30) A is changed to 0 and B to 1.
- Inputs to a 3-bit truth table can be generated with the initial block

```
initial
begin
    D = 3'b000;
    repeat (7);
    #10    D = D + 3'b001;
end
```

- The 3-bit vector D is initialized to 000 at time=0. The keyword **repeat** specifies looping statement: one is added to D seven times, once every 10 time units.

Writing a Test-Bench (2)

- A stimulus module is an HDL program that has the following form.

module testname

Declare local **reg** and **wire** identifiers

Instantiate the design module under test.

Generate stimulus using **initial** and **always** statements

Display the output response.

endmodule

- A test module typically has no inputs or outputs.
- The signals that are applied as inputs to the design module for simulation are declared in the stimulus module as local reg data type.
- The outputs of the design module that are displayed for testing are declared in the stimulus model as local wire data type.
- The module under test is then instantiated using the local identifiers.

Writing a Test-Bench (3)

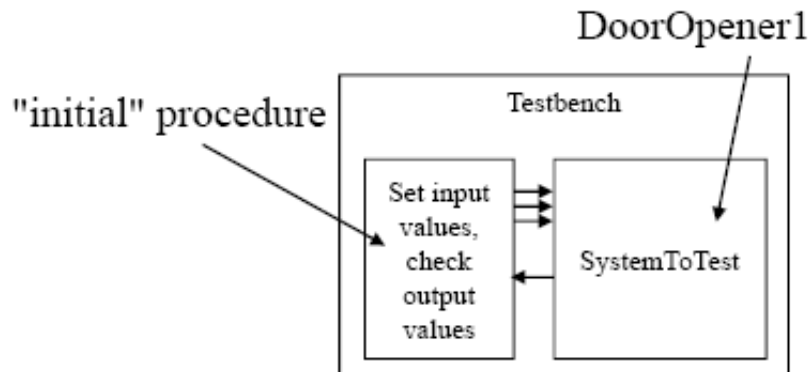
- The response to the stimulus generated by the initial and always blocks will appear at the output of the simulator as timing diagrams.
- It is also possible to display numerical outputs using Verilog *system tasks*.
 - \$display – display one-time value of variables or strings with end-of-line return,
 - \$write – same \$display but without going to next line.
 - \$monitor – display variables whenever a value changes during simulation run.
 - \$time – displays simulation time
 - \$finish – terminates the simulation
- The syntax for \$display,\$write and \$monitor is of the form
Task-name (format-specification, argument list);
E.g. \$display(%d %b %b, C,A,B);
\$display("time = %d A = %b B=%b",\$time,A,B);

Testbench in Verilog

Module

- Three register signals for inputs
(values must be written and stored)
- One wire signal for output (value is only read, need not be stored)
- Instantiates component, connects to signals
- "initial" procedure executed only once at beginning
 - Sets input values
 - Displays output value

```
module Testbench;  
    reg c, h, p;  
    wire f;  
  
    DoorOpener DoorOpener1(c, h, p, f);  
  
    initial  
    begin  
        // case 0  
        c <= 0; h <= 0; p <= 0;  
        #1 $display("f = %b", f);  
        // case 1  
        c <= 0; h <= 0; p <= 1;  
        #1 $display("f = %b", f);  
        // (cases 2-6 omitted from figure)  
        // case 7  
        c <= 1; h <= 1; p <= 1;  
        #1 $display("f = %b", f);  
    end  
endmodule
```



Writing a Test-Bench (4)

```
//Stimulus for mux2x1_df.  
module testmux;  
    reg TA,TB,TS; //inputs for mux  
    wire Y;       //output from mux  
    mux2x1_df mx (TA,TB,TS,Y); // instantiate mux  
    initial  
        begin  
            TS = 1; TA = 0; TB = 1;  
            #10 TA = 1; TB = 0;  
            #10 TS = 0;  
            #10 TA = 0; TB = 1;  
        end  
    initial  
        $monitor("select = %b A = %b B = %b OUT = %b time = %0d",  
            TS, TA, TB, Y, $time);  
endmodule
```

Writing a Test-Bench (5)



```
//Description of 2-to-1-line multiplexer  
module mux2x1_df (A,B,select,OUT);  
    input A,B,select;  
    output OUT;  
    assign OUT = select ? A : B;  
endmodule
```

Output Format Strings

- Formatting string

- %h, %H hex
- %d, %D decimal
- %o, %O octal
- %b, %B binary
- %t time

- `$monitor("%t: %b %h %h %h %b\n",
$time, c_out, sum, a, b, c_in);`

- Can get more details from Verilog standard

Example

*not an
apostrophe!*

```
timescale 1ns /1ns // time unit/time precision
module t_adder4b;
  reg[8:0] stim; // inputs to UUT are regs
  wire[3:0] S; // outputs of UUT are wires
  wire C4;
  // instantiate UUT
  adder4b(S, C4, stim[8:5], stim[4:1], stim[0]);
  // stimulus generation
  initial begin
    stim = 9'b0000000000; // at 0 ns
    #10 stim = 9'b111100001; // at 10 ns
    #10 stim = 9'b000011111; // at 20 ns
    #10 stim = 9'b111100010; // at 30 ns
    #10 stim = 9'b000111110; // at 40 ns
    #10 $stop; // at 50 ns
  end
endmodule
```

*all inputs grouped
into single vector
(not required)*

*see "response"
to each of these
input vectors*

stops simulation

*timing control
for simulation*

*Behav.
Verilog:
"do this
once"*

Output Example

```
`timescale 1ns /1ns                // time_unit/time_precision
module t_adder4b;
    reg[8:0] stim;                    // inputs to UUT are regs
    wire[3:0] S;                      // outputs of UUT are wires
    wire C4;

    // instantiate UUT
    adder4b(S, C4, stim[8:5], stim[4:1], stim[0]);

    // monitor statement
    initial $monitor($time, C4, S, stim[8:5], stim[4:1], stim[0]);
    // stimulus generation
    initial begin
        stim = 9'b0000000000;        // at 0 ns
        #10 stim = 9'b111100001;    // at 10 ns
        #10 stim = 9'b000011111;    // at 20 ns
        #10 stim = 9'b111100010;    // at 30 ns
        #10 stim = 9'b000111110;    // at 40 ns
        #10 $stop;                   // at 50 ns – stops simulation
    end
endmodule
```

All values will run together, easier to read with formatting string



Exhaustive Testing

- For combinational designs w/ up to 8 or 9 inputs
 - Test ALL combinations of inputs to verify output
 - Could enumerate all test vectors, but don't...
 - Generate them using a "for" loop!

```
reg [4:0] x;  
initial begin  
    for (x = 0; x < 16; x = x + 1)  
        #5 // need a delay here!  
end
```

- Need to use "reg" type for loop variable? Why?

Why Loop Vector Has Extra Bit

- Want to test all vectors 0000 to 1111

```
reg [3:0] x;
```

```
initial begin
```

```
    for (x = 0; x < 16; x = x + 1)
```

```
        #5    // need a delay here!
```

```
end
```

- If x is 4 bits, it only gets up to 1111 => 15

➤ 1100 => 1101 => 1110 => 1111 => 0000 => 0001

- x is never ≥ 16 ... so loop goes forever!

Example: UUT



```
module Comp_4_str(A_gt_B, A_lt_B, A_eq_B, A, B);  
output      A_gt_B, A_lt_B, A_eq_B;  
input [3:0] A, B;  
    // Code to compare A to B  
    // and set A_gt_B, A_lt_B, A_eq_B accordingly  
endmodule
```

Example: Testbench

```
module t_Comp_4_str();
wire    A_gt_B, A_lt_B, A_eq_B;
reg      [4:0] A, B;           // sized to prevent loop wrap around
wire     [3:0] A_bus, B_bus;
assign   A_bus = A[3:0];      // display 4 bit values
assign   B_bus = B[3:0];

Comp_4_str M1 (A_gt_B, A_lt_B, A_eq_B, A[3:0], B[3:0]);    // UUT

initial $monitor("%t A: %h B: %h AgtB: %b AltB: %b AeqB: %b",
    $time, A_bus, B_bus, A_gt_B, A_lt_B, A_eq_B);
initial #2000 $finish;      // end simulation, quit program

initial begin
#5 for (A = 0; A < 16; A = A + 1) begin // exhaustive test of valid inputs
    for (B = 0; B < 16; B = B + 1) begin #5; // may want to test x's and z's
        end // first for
    end // second for
end // initial
endmodule
```

note multiple initial blocks

Generating Clocks

❑ Wrong way:

```
initial begin
    #5 clk = 0;
    #5 clk = 1;
    #5 clk = 0;
    ... (repeat hundreds of times)
end
```

❑ Right way:

```
initial clk = 0;
always @(clk)
    clk = #5 ~clk;
```

```
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
```

❑ LESS TYPING

❑ Easier to read, harder to make mistake

Next Time

- FPGAs
- First research paper to read will be out (to be discussed on 10/6)

Questions?